Group No.:5

Group Members:	Ayushi Ray	**PRN no.:** 24070521145	**SEM:** 4th C
	Shruti Bawankar	24070521148	4th C
	Radhika Chaudhari	24070521150	4th C

Preprocessing Student Queries

Problem Statement

Introduction

With the rapid growth of conversational systems and intelligent assistants, Natural Language Processing (NLP) has become an essential technology for understanding human language. In educational environments, students frequently ask questions related to examinations, courses, fees, and schedules. However, these queries are often written in informal language, containing spelling errors, abbreviations, punctuation, and unnecessary words.

To enable machines to correctly interpret such queries, **text preprocessing** is required. Text preprocessing is the process of converting raw textual input into a clean and structured format suitable for further processing by machine learning models or chatbot systems.

This project implements a **text preprocessing pipeline** designed specifically for student questions. The pipeline performs several steps including **lowercasing, spelling normalization, punctuation removal, tokenization, stopword removal, spell checking, lemmatization, and sentence reconstruction**.

Objectives

The objectives of this project are:

1. To clean and normalize student queries written in natural language.
2. To remove unnecessary words and punctuation from the text.
3. To correct spelling errors in user queries.
4. To convert words into their base grammatical form.
5. To generate a clean sentence that can be easily used by a chatbot or machine learning model.

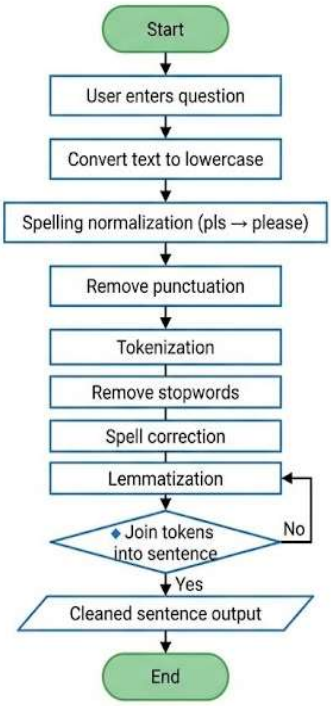
Methodology

The system processes student queries through a **sequence of preprocessing steps**. Each step improves the quality of the input text.

The main stages of the preprocessing pipeline are:

1. User input collection
2. Lowercase conversion
3. Spelling normalization
4. Punctuation removal
5. Tokenization
6. Stopword removal
7. Spell correction
8. Lemmatization
9. Sentence reconstruction

Flowchart:



Technologies Used:

Tools and Technologies Used

Python - Main programming language used for implementation

NLTK - Used for tokenization, stopword removal, POS tagging, and lemmatization

PySpellChecker - Used to detect and correct spelling mistakes

Regular Expressions - Used for pattern-based text replacement

Jupyter Notebook - Development environment

Explanation of Each Preprocessing Step

1 Lowercase Conversion

The first step converts the entire input text to lowercase.

Example:

Original: When is the Exam for SEM 5?
Lowercase: when is the exam for sem 5?

This ensures that the system treats words like Exam and exam as the same.

2 Spelling Normalization

Students often use informal abbreviations when typing queries.

Example:

Informal Word	Normalized Word
pls	please
plz	please
u	you
ur	your
thx	thanks

Using regular expressions, these words are replaced with their proper form before further processing.

3 Punctuation Removal

Punctuation symbols do not usually contribute to meaning in simple queries.

Example:

Original: fee's structure & timings!!!
Processed: fees structure timings

The punctuation characters are removed using Python's string translation function.

4 Tokenization

Tokenization splits the sentence into individual words called tokens.

Example:

Sentence:
please tell fee structure timing

Tokens:
['please', 'tell', 'fee', 'structure', 'timing']

Tokenization is performed using the tokenizer provided by NLTK.

5 Stopword Removal

Stopwords are commonly used words that carry little meaning in a sentence.

Examples include:

is, the, am, are, was, were, a, an, and

Example:

Input Tokens:
['please', 'tell', 'me', 'the', 'fee', 'structure']

After Stopword Removal:
['please', 'tell', 'fee', 'structure']

Removing stopwords helps focus on meaningful words.

6 Spell Correction

Students may make typing mistakes while entering queries.

Example:

timngs → timings
strcture → structure

The system uses PySpellChecker to automatically correct spelling errors.

7 Lemmatization

Lemmatization converts words into their root or dictionary form.

Example:

Word	Lemma
running	run
timings	timing
exams	exam

Lemmatization improves text consistency and helps NLP models better understand language patterns.

This process is performed using WordNet Lemmatizer from NLTK.

8 Sentence Reconstruction

After preprocessing, the tokens are joined back into a clean sentence.

Example:

Tokens:
['please', 'tell', 'fee', 'structure', 'timing']

Clean Sentence:
"please tell fee structure timing"

This cleaned sentence can now be used for:

- Chatbot response matching
- Machine learning models
- FAQ systems
- Intent classification

Sample Input and Output

Raw Student Query =	Processed Output
Pls tell me fee's structure & timings!!!=	please tell fee structure timing
When is SEM 5 CS exam?=	sem 5 cs exam
wht is exam timng?=	exam timing

Advantages of the Proposed System

1. Improves understanding of informal student queries.
2. Removes unnecessary noise from text.
3. Corrects spelling errors automatically.

4. Standardizes words using lemmatization.
5. Produces structured input suitable for chatbot systems.

Applications

The preprocessing pipeline can be used in:

- University chatbot systems
- FAQ automation systems
- Educational virtual assistants
- AI-based student help desks
- Question classification systems

This ensures the conversation continues smoothly.

Limitations

Despite its effectiveness, the system has certain limitations:

1. Some spelling corrections may be incorrect for domain-specific words.
2. Context understanding is limited without advanced NLP models.
3. Abbreviation dictionary must be manually updated.

Future Enhancements

The system can be further improved by:

1. Adding Named Entity Recognition (NER) to detect course codes and semester numbers.
2. Integrating TF-IDF vectorization for machine learning models.
3. Implementing intent classification using algorithms like Logistic Regression or Support Vector Machines.
4. Expanding the abbreviation dictionary.

Output Screenshots:

```
[3]: #Initailizing objects
     stop_words = set(stopwords.words('english'))
     lemmatizer = WordNetLemmatizer()
     spell = SpellChecker()

[4]: SPELLING_MAP = {
         "pls": "please",
         "plz": "please",
         "u": "you",
         "ur": "your",
         "thx": "thanks"
     }

[5]: def get_wordnet_pos(word):
         tag = nltk.pos_tag([word])[0][1][0].upper()
         tag_dict = {
             'J': wordnet.ADJ,
             'N': wordnet.NOUN,
             'V': wordnet.VERB,
             'R': wordnet.ADV
         }
         return tag_dict.get(tag, wordnet.NOUN)

[6]: def preprocess_question(text):
         # Lowercase
         text = text.lower()

         # Spelling normalization
         for wrong, correct in SPELLING_MAP.items():
             text = re.sub(r'\b' + wrong + r'\b', correct, text)

         # Remove punctuation
         text = text.translate(str.maketrans('', '', string.punctuation))
```

```
         # Remove punctuation
         text = text.translate(str.maketrans('', '', string.punctuation))

         # Tokenization
         tokens = word_tokenize(text)

         # Stopword removal + spell correction
         corrected_tokens = []
         for word in tokens:
             if word not in stop_words:
                 corrected_word = spell.correction(word)
                 corrected_tokens.append(corrected_word if corrected_word else word)

         # Lemmatization
         lemmatized_tokens = [
             lemmatizer.lemmatize(word, get_wordnet_pos(word))
             for word in corrected_tokens
         ]

         return lemmatized_tokens

[7]: user_input = input("Enter your question: ")
     output = preprocess_question(user_input)
     print("Processed Output:", output)

     Enter your question:  Plz tell the timings!!
     Processed Output: ['please', 'tell', 'timing']

[8]: clean_sentence = " ".join(output)
     print(clean_sentence)

     please tell timing

[ ]:
```

Conclusion

This project demonstrates the implementation of an effective text preprocessing pipeline for student queries. The system successfully cleans and standardizes raw text by applying several NLP techniques such as tokenization, stopword removal, spell correction, and lemmatization.

By converting informal and noisy queries into structured text, the preprocessing pipeline significantly improves the performance of chatbot systems and machine learning models. This approach can serve as a foundational component for building intelligent academic assistance systems.